

UNIVERSIDADE DE BRASÍLIA
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

116394 – ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES

Relatório de Implementação

Geração de Dados Imediatos no RISC-V

Aluno: Victor Enéias

Matrícula: 221038364

Turma: Turma 2

Professor: RICARDO JACOBI

1. Objetivo

O objetivo deste trabalho foi implementar, em VHDL, um módulo capaz de gerar o dado imediato de uma instrução RISC-V de 32 bits.

O módulo desenvolvido recebe como entrada uma instrução completa, representada pelo sinal `instr`, e produz como saída o imediato correspondente, representado pelo sinal `imm32`. A saída possui 32 bits e é declarada como do tipo `signed`.

A entidade implementada foi:

```
entity genImm32 is
  port (
    instr : in  std_logic_vector(31 downto 0);
    imm32 : out signed(31 downto 0)
  );
end entity genImm32;
```

Foram considerados os formatos R, I, I*, S, SB, U e UJ do conjunto de instruções RISC-V.

2. Descrição geral da implementação

A implementação foi feita por meio da análise do campo `opcode`, localizado nos bits 6 `downto 0` da instrução.

No código VHDL, o campo `opcode` é extraído da seguinte forma:

```
opcode := instr(6 downto 0);
```

A partir desse valor, foi utilizada uma estrutura `case` para identificar o formato da instrução e gerar o imediato correspondente.

Os opcodes considerados foram:

- 0x33: formato R;
- 0x03: formato I, usado em instruções de load;
- 0x13: formato I, usado em operações imediatas;
- 0x67: formato I, usado em `jalr`;
- 0x23: formato S;
- 0x63: formato SB;
- 0x37: formato U;
- 0x6F: formato UJ.

Para o formato R, que não possui imediato, a saída gerada é zero.

```
when "0110011" =>  
    imm32 <= to_signed(0, 32);
```

3. Geração dos imediatos

3.1 Formato R

O formato R não possui campo imediato. Portanto, para instruções desse tipo, o módulo gera o valor zero.

```
imm32 <= to_signed(0, 32);
```

3.2 Formato I

No formato I, o imediato possui 12 bits e ocupa os bits 31 `downto` 20 da instrução.

Como esse imediato pode representar valores negativos, é necessário aplicar extensão de sinal para gerar o resultado final de 32 bits.

A implementação foi:

```
imm32 <= resize(signed(instr(31 downto 20)), 32);
```

A função `resize` foi usada para expandir o valor de 12 bits para 32 bits, preservando o sinal.

3.3 Formato I*

O formato I* é usado em instruções de deslocamento, como `srai`. Nesse caso, o imediato representa o campo `shamt`, ou seja, a quantidade de posições do deslocamento.

Diferentemente do formato I comum, o campo usado é apenas:

$$instr(24\text{downto}20)$$

Assim, o imediato é gerado com zeros nos bits superiores.

A implementação foi:

```
imm32 <= to_signed(to_integer(unsigned(instr(24 downto 20))), 32);
```

Essa conversão garante que o campo `shamt` seja tratado como um valor sem sinal.

3.4 Formato S

No formato S, o imediato é dividido em duas partes dentro da instrução:

$$imm(11 : 5) = instr(31 : 25)$$

$$imm(4 : 0) = instr(11 : 7)$$

Portanto, essas duas partes precisam ser concatenadas.

A implementação foi:

```
imm32 <= resize(signed(instr(31 downto 25) & instr(11 downto 7)), 32);
```

Como o imediato do formato S pode ser negativo, também é aplicada extensão de sinal.

3.5 Formato SB

O formato SB é usado em instruções de desvio condicional, como `beq` e `bne`.

Nesse formato, os bits do imediato aparecem embaralhados dentro da instrução. O imediato é montado da seguinte forma:

$$imm(12 : 1) = \{instr(31), instr(7), instr(30 : 25), instr(11 : 8)\}$$

Além disso, o bit menos significativo é sempre zero:

$$imm(0) = 0$$

A implementação foi:

```
imm32 <= resize(  
    signed(instr(31) & instr(7) & instr(30 downto 25) & instr(11 downto 8) &  
    '0'),  
    32  
);
```

A extensão de sinal é feita a partir do bit mais significativo do imediato, que corresponde ao bit `instr(31)`.

3.6 Formato U

No formato U, o imediato ocupa os bits superiores da instrução:

$$imm(31 : 12) = instr(31 : 12)$$

Os 12 bits inferiores são preenchidos com zero:

$$imm(11 : 0) = 0$$

A implementação foi:

```
imm32 <= signed(instr(31 downto 12) & "000000000000");
```

Esse formato é usado, por exemplo, pela instrução `lui`.

3.7 Formato UJ

O formato UJ é usado pela instrução `jal`. Assim como no formato SB, os bits do imediato aparecem reorganizados dentro da instrução.

O imediato é montado da seguinte forma:

$$imm(20 : 1) = \{instr(31), instr(19 : 12), instr(20), instr(30 : 21)\}$$

E o bit menos significativo é sempre zero:

$$imm(0) = 0$$

A implementação foi:

```
imm32 <= resize(
    signed(instr(31) & instr(19 downto 12) & instr(20) & instr(30 downto 21) &
        '0'),
    32
);
```

4. Testbench

Para verificar o funcionamento do módulo, foi criado um testbench chamado `tb_genImm32.vhd`.

O testbench instancia o módulo `genImm32`, aplica instruções RISC-V conhecidas na entrada e compara a saída gerada com o valor esperado.

A verificação foi feita com comandos `assert`. Caso algum valor esteja incorreto, a simulação emite uma mensagem de erro.

A procedure usada para testar cada instrução foi:

```
procedure check_imm(
    constant nome      : in string;
    constant codigo    : in std_logic_vector(31 downto 0);
    constant esperado  : in signed(31 downto 0)
) is
begin
    instr <= codigo;
    wait for 10 ns;

    assert imm32 = esperado
        report "ERRO em " & nome
        severity error;

    report "OK: " & nome
        severity note;
end procedure;
```

Foram testadas as instruções indicadas no enunciado, cobrindo os formatos R, I, I*, S, SB, U e UJ.

5. Casos de teste

A tabela a seguir apresenta os casos verificados no testbench.

Instrução	Código	Formato	Imediato esperado
add t0, zero, zero	0x000002B3	R	0
lw t0, 16(zero)	0x01002283	I	16
addi t1, zero, -100	0xF9C00313	I	-100
xori t0, t0, -1	0xFFFF2C293	I	-1
addi t1, zero, 354	0x16200313	I	354
jalr zero, zero, 0x18	0x01800067	I	24
srai t1, t2, 10	0x40A3D313	I*	10
lui s0, 2	0x00002437	U	8192
sw t0, 60(s0)	0x02542E23	S	60
bne t0, t0, main	0xFE5290E3	SB	-32
jal rot	0x00C000EF	UJ	12

6. Comandos de simulação

A simulação pode ser realizada com o GHDL usando os seguintes comandos:

```
ghdl -a --std=08 genImm32.vhd
ghdl -a --std=08 tb_genImm32.vhd
ghdl -e --std=08 tb_genImm32
ghdl -r --std=08 tb_genImm32
```

Caso seja desejado gerar um arquivo de formas de onda, pode-se usar:

```
ghdl -r --std=08 tb_genImm32 --vcd=ondas.vcd
gtkwave ondas.vcd
```

7. Respostas às perguntas do relatório

7.1 Qual a razão do embaralhamento dos bits do imediato no RISC-V?

O embaralhamento dos bits do imediato no RISC-V ocorre por causa do formato fixo das instruções de 32 bits e da tentativa de manter certos campos sempre nas mesmas posições.

Campos como `opcode`, `rd`, `rs1`, `rs2` e `funct3` aparecem em posições fixas ou semelhantes nos diferentes formatos. Isso simplifica o hardware de decodificação, pois o processador consegue encontrar os registradores e o opcode sempre nos mesmos lugares.

Como consequência, nos formatos que precisam de imediato, os bits do imediato são distribuídos nos espaços que sobram da instrução. Por isso, em formatos como SB e UJ, o imediato aparece aparentemente embaralhado.

Além disso, o bit de sinal dos imediatos fica no bit 31 da instrução em vários formatos. Isso facilita a extensão de sinal, pois o hardware pode usar sempre o mesmo bit da instrução como referência para determinar se o imediato é positivo ou negativo.

Portanto, o embaralhamento não é aleatório. Ele é uma escolha de projeto para simplificar a decodificação dos campos principais da instrução e reduzir a complexidade do hardware.

7.2 Por que alguns imediatos não incluem o bit 0?

Alguns imediatos, como os usados nos formatos SB e UJ, representam deslocamentos de endereço para instruções de desvio ou salto.

Nesses casos, o destino do salto costuma estar alinhado na memória. Como as instruções possuem alinhamento, o bit menos significativo do endereço de destino é sempre zero. Portanto, não é necessário armazenar esse bit dentro da instrução.

Em vez de gastar um bit da instrução armazenando um valor que sempre será zero, o RISC-V omite esse bit e o módulo gerador de imediato o recoloca manualmente como zero.

No VHDL, isso foi feito concatenando o caractere '0' ao final do imediato:

```
... & '0'
```

Assim, o imediato gerado já representa corretamente o deslocamento usado no cálculo do endereço de destino.

7.3 Os imediatos de operações lógicas estendem o sinal?

Sim. As operações lógicas imediatas do tipo I, como `xori`, `ori` e `andi`, usam imediato de 12 bits com extensão de sinal.

Isso significa que, se o bit mais significativo do imediato de 12 bits for 1, os bits superiores do imediato de 32 bits serão preenchidos com 1. Se o bit mais significativo for 0, os bits superiores serão preenchidos com 0.

Um exemplo importante é a instrução:

```
xori t0, t0, -1
```

O imediato -1, em 12 bits, é representado como:

111111111111

Após a extensão de sinal para 32 bits, esse valor se torna:

0xFFFFFFFF

Portanto, mesmo em operações lógicas, o imediato é estendido com sinal. A exceção são as instruções de deslocamento, como `slli`, `srl` e `srai`, que usam o campo `shamt` como quantidade de deslocamento e não como um imediato assinado comum.

7.4 Como é implementada a instrução NOT no RISC-V?

O RISC-V não possui uma instrução básica específica chamada `not`. Em vez disso, `not` é implementada como uma pseudoinstrução.

A pseudoinstrução:

```
not rd, rs
```

é convertida pelo montador para:

```
xori rd, rs, -1
```

Isso funciona porque a operação XOR com 1 inverte um bit:

$$0 \oplus 1 = 1$$

$$1 \oplus 1 = 0$$

Como o imediato `-1` é estendido para `0xFFFFFFFF`, todos os bits do registrador são comparados com 1. Assim, todos os bits são invertidos, produzindo o comportamento da operação `not`.

Portanto:

$$\text{not } rd, rs \equiv \text{xori } rd, rs, -1$$

8. Conclusão

Neste trabalho foi implementado um módulo VHDL para geração de dados imediatos em instruções RISC-V.

O módulo analisa o opcode da instrução e, de acordo com o formato identificado, reconstrói o imediato correspondente. Foram tratados os formatos R, I, I*, S, SB, U e UJ. Também foram aplicadas corretamente a extensão de sinal e a inserção de bits zero nos formatos que exigem alinhamento.

O testbench desenvolvido verifica todos os casos fornecidos no enunciado, comparando a saída do módulo com os imediatos esperados. Com isso, a implementação permite validar o funcionamento do gerador de imediatos de forma automatizada.

9. Arquivos entregues

A entrega será realizada em um arquivo compactado identificado com o número de matrícula do aluno. O arquivo compactado conterá:

- código VHDL do módulo principal: `genImm32.vhd`;
- testbench: `tb_genImm32.vhd`;
- relatório de implementação: `relatorio_genImm32.pdf`.